Simple formulas, room for custom models

BayesianRegressionModels.jl: regression formulas and custom model code together

BayesianRegressionModels.jl

2026-09-14

Start with a familiar regression

USE FORMULAS WHERE THEY HELP

```julia
gaussian = (@brm begin
    sigma ~ Exponential(2)
    mu ~ 1 + x
    effect(:, :) ~ Normal(0, 2)
    effect(mu, x) ~ Normal(0, 0.25)
    y ~ Normal(mu, sigma)
end)((; x=[-1.0, 0.5, 2.0], y=[0.2, 1.1, -0.4]))
```

The formula handles the regression.

`mu ~ 1 + x` gives an intercept and a slope.

`effect(...)` assigns priors to those coefficients.

The noise parameter and observation distribution are written alongside them.

> The aim: keep this convenience when a model needs more than regression.

Existing executable example from the feature atlas. Julia setup: `using BayesianRegressionModels, Distributions`.

Add the part that describes your science

EXAMPLE: DRUG CONCENTRATION IN EACH SUBJECT

```
sigma ~ Exponential(1)
log(CL) ~ 1 + (1 | pk | subject)
log(V)  ~ 1 + (1 | pk | subject)
samples = groups(obs_subject => subject)

@plate for i in eachindex(CL)
    predicted_concentration[i] = dose[i] / V[i] .*
        exp.((-CL[i] / V[i]) .* time[samples[i]])
    concentration[samples[i]] ~ Normal(predicted_concentration[i],
 sigma)
end
```

Formula lines: describe how clearance (`CL`) and volume (`V`) vary between subjects.

Custom calculation: the `@plate for` cell computes each subject's concentration curve.

Observation line: compares that curve with measured concentrations.

The shared `pk` label allows clearance and volume effects to be correlated.

Formulas and a custom calculation live in the same model declaration.

StanBlocks example. Body of the existing `@brm begin … end` declaration; complete code and data in appendix B. `@plate for` runs the calculation once per subject.

Subjects and measurements can fit together

THE SAME PK EXAMPLE

```
# Subject axis: one row per subject.
subject=["alice", "bob"],
dose=[100.0, 80.0],

# Observation axis: one row per sample, interleaved by subject.
obs_subject=["alice", "bob", "alice", "bob", "alice"],
time=[0.5, 0.25, 1.5, 1.0, 3.0],
concentration=[8.1, 7.6, 5.2, 4.9, 2.1],
```

Two subjects, five measurements.

Each subject has a dose, clearance, and volume. Each measurement has a time and a subject label.

`ragged(time, obs_subject)` groups the times for the calculation. The response uses the same grouping.

Describe subject differences with formulas; evaluate the curve at each subject’s measurement times.

Data fields passed to the declaration on the previous slide. The interleaved rows are intentional.

The same approach reaches larger models

EXISTING BRM EXAMPLES WITH CUSTOM SCIENTIFIC CALCULATIONS

Warfarin PK/PD

Subject effects feed concentration calculations and an ODE for drug response.

Full equations, code, and audit

StanBlocks

Public two-stage model: compilation and finite density/gradient checks. A joint extension is documented separately.

Wastewater

Regression and group effects combine with infection renewal, shedding, and measurement models.

Formula and calculation definitions

StanBlocks

Structural port: compilation and finite density/gradient checks; not a numerically equivalent CDC reproduction.

Grey-seal population

Covariate and group effects combine with a population process and eight observation streams.

Formula and population-process code

StanBlocks

Compilation checks on an example dataset; no completed posterior fit.

Keep the useful regression formulas as the scientific calculation becomes more involved.

Compare the same modeling task

SUBJECT EFFECTS + A CUSTOM CONCENTRATION CURVE

APPROACH	REGRESSION AND SUBJECT EFFECTS	CUSTOM SCIENTIFIC CALCULATION
brms	Formula-based regression and nonlinear parameter models.	Nonlinear expressions and custom Stan functions; custom families have additional prediction and likelihood hooks.
BRM	Formula-based regression and subject effects inside the model block.	The PK example passes their values directly into a custom calculation, then uses its result in the likelihood.
Hand-written Stan or Turing	Write the coefficients, group distributions, and predictor calculations explicitly.	Write the scientific calculation in the modeling language and its supported libraries.

> BRM’s attraction is the combination: concise regression code with room for the rest of the model.

brms nonlinear models · custom Stan code · Turing with differential equations. This compares authoring approaches, not measured development time or fitting speed.

Custom code has specific requirements

FLEXIBILITY IN PRACTICE

A Julia function or distribution

RKBRMI emits ordinary Julia calculations into RKPPL source.

It still needs the operations used by the fit: a valid density, a supported gradient path, and sampling support when generating predictions.

A custom Stan calculation

StanBlocks needs code it can translate to Stan.

A calculation written only for Stan also needs a native implementation for RKBRMI.

> Adding a calculation and adding a reusable model term are different amounts of work.

A new parameter structure or fitted transform may need extra methods. See the current extension contracts.

A different parameterization can help sampling

SAME PROBABILITY MODEL, DIFFERENT VARIABLES FOR THE SAMPLER

Sample the coefficient directly

$$\beta_k \sim \mathcal{N}(0, s_k)$$

Often useful when the data strongly identify the coefficient.

Sample a scaled coefficient

$$z_k \sim \mathcal{N}(0, 1), \qquad \beta_k = s_k z_k$$

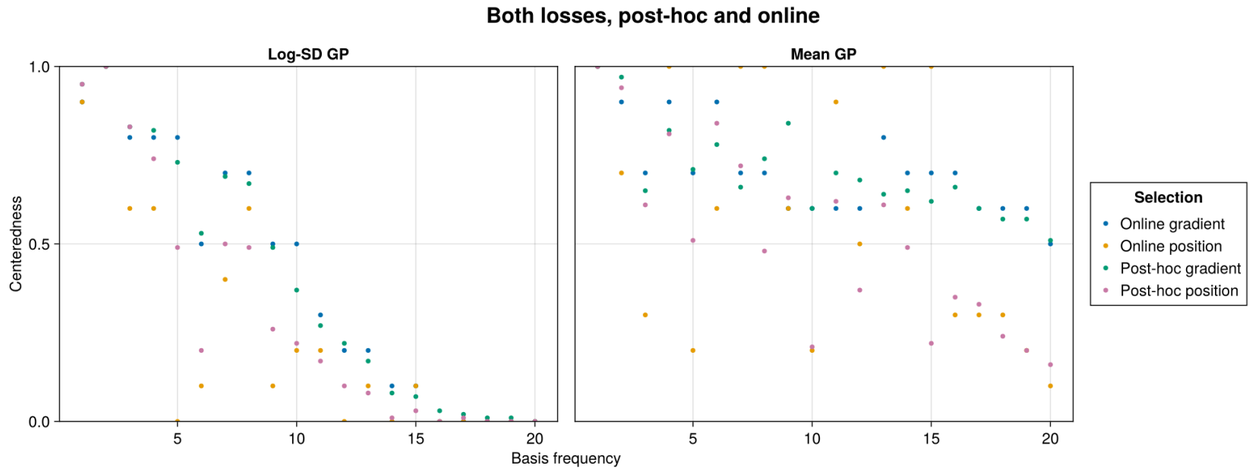
Often useful when the data provide less information.

> BRM’s HSGP example can choose between these forms separately for each coefficient.

The example models motorcycle-impact acceleration over time, with a Gaussian process for the mean and another for the noise level. It uses all 133 observations and 20 basis functions per process.

One model, six recorded fits

REFRESHED HSGP STUDY · STANBLOCKS SAMPLING



Each coefficient gets its own choice. Zero is fully noncentered; one is fully centered. The two losses and the two procedures need not agree.

FIT	DIVERGENCES	SAMPLING EFFICIENCY	TOTAL EFFICIENCY
NCP	99	1×	1×
CP	707	0.0227×	0.0236×
Post-hoc position	55	5.67×	0.934×
Post-hoc gradient	43	6.25×	1.04×
Online position	6	1.93×	3.43×
Online gradient	12	6.23×	7.15×

One chain per fit. These runs do not establish a reliable ranking or a general speed advantage. Split R-hat compares parts of the same chain.

Efficiency relative to NCP. Minimum bulk ESS over 192 scientific quantities per sampling gradient, or per gradient of the whole workflow. Post-hoc totals include the pilot.

Make predictions with the fitted model

KEEP THE MEANING OF THE REGRESSION WHEN THE DATA CHANGE

During fitting

Store the category levels, scaling constants, spline knots, and GP basis used to construct the model.

For new measurements

Apply those same transformations to new inputs, so the posterior coefficients still have the same meaning.

For new subjects

Choose explicitly how new group effects are generated. Check that the model and execution path support the requested prediction.

> Custom model components should fit into the prediction workflow as well as the model declaration.

Current operations have documented limits; this is not a claim that every custom component already supports every prediction task.

A possible future: Julia models on GPUs

A CONCRETE DIRECTION TO INVESTIGATE

The compiler path exists

Reactant compiles Julia computations through MLIR and XLA for CPU, GPU, and TPU execution.

It also documents a probabilistic-programming interface, including custom log-density functions.

BRM integration remains to be shown

A possible next step is to compile suitable Julia model code through this path.

Compatibility, gradients, and useful performance need to be demonstrated on actual BRM models.

> Keep the model declaration useful as more of its computation can run on accelerators.

Reactant · probabilistic programming · control-flow requirements. brms already exposes Stan/OpenCL support for some operations; GPU access alone is not the proposed distinction.

Simple formulas, room for custom models

END OF MAIN TALK

Use concise formulas for the regression parts, and combine them with custom model code when the science needs more.

The PK example shows the idea today. The larger examples show where it can go—and which execution and prediction details still need work.

Current execution support

APPENDIX A

MODEL COMPONENT	EXECUTION	CURRENT BOUNDARY
Population and distributional regression	StanBlocks + RK(PPL)	Shared formulas, links, and prior addresses.
Group effects, advanced terms, modified responses	Both, within limits	Individual features and their combinations have specific supported subsets.
Ordinary Julia callables without a Stan translation	RK(PPL)	Density, gradient, transform, and prediction requirements still apply.
Custom structural code written for StanBlocks	StanBlocks	A native implementation is needed for RK(PPL).
Corrected HSGP centering study	StanBlocks sampled	See the study-specific checks in appendix E.

BRM has two backends: StanBlocks and RK(PPL). Their supported operations and generated representations are checked separately.

The complete PK example

APPENDIX B

```
population_pk = (@brm begin
    sigma ~ Exponential(1)
    log(CL) ~ 1 + (1 | pk | subject)
    log(V)  ~ 1 + (1 | pk | subject)
    samples = groups(obs_subject => subject)

    @plate for i in eachindex(CL)
        predicted_concentration[i] = dose[i] / V[i] .*
            exp.((-CL[i] / V[i]) .* time[samples[i]])
        concentration[samples[i]] ~ Normal(predicted_concentration[i], sigma)
    end
end)((;
    # Subject axis: one row per subject.
    subject=["alice", "bob"],
    dose=[100.0, 80.0],

    # Observation axis: one row per sample, interleaved by subject.
    obs_subject=["alice", "bob", "alice", "bob", "alice"],
    time=[0.5, 0.25, 1.5, 1.0, 3.0],
    concentration=[8.1, 7.6, 5.2, 4.9, 2.1],
))
```

StanBlocks example. Julia setup: `using BayesianRegressionModels, Distributions`. Code and data are read from the executable feature atlas.

Inspect the actual model code

APPENDIX C

Start from the declaration

The feature atlas executes each displayed model.

Its four views show the BRM declaration, intermediate representation, emitted StanBlocks code, and generated Stan.

Check the operation you need

A model that can evaluate a density may still lack a prediction method or a required gradient.

For new fitted terms, check new-data transformations too. For a second execution path, check the same statistical model and parameter mapping.

> Read generated code from the executable example, not from a manually maintained illustration.

How the comparison is generated · Extension contracts

The centering study in detail

APPENDIX D

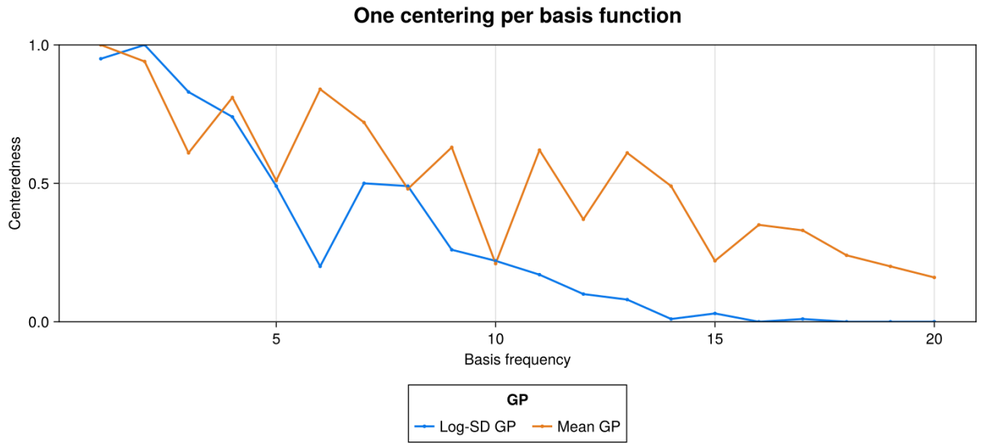
Mean and noise both vary with time

$$\begin{aligned} \mu(t) &= \mathrm{HSGP}_{\mu}(t), \\ \log \sigma(t) &= \mathrm{HSGP}_{\sigma}(t), \\ y(t) &\sim \mathcal{N}(\mu(t), \sigma(t)). \end{aligned}$$

Each GP uses 20 basis functions. Its log length scale and log marginal standard deviation have $\mathcal{N}(0, 4)$ priors.

Partial centering

$$u_k \sim \mathcal{N}(0, s_k^{c_k}), \qquad \beta_k = s_k^{1-c_k} u_k.$$



1 noncentered pilot **2** choose each c_k **3** fresh partial fit

This source workflow is the refreshed study's post-hoc position fit. CP, the gradient loss and online adaptation are BRM extensions.

What the centering results establish

APPENDIX E

Recorded evidence

- All 133 rows; 20 basis functions per GP.
- Seed 1; 10,000 retained draws per fit; ordinary WarmupHMC initialization and adaptation.
- Original Stan density and gradient checks: 26 for the noncentered target; 51 for the partial target.
- All 40 post-hoc position choices match the source reproduction’s selection.
- Saved-draw mapping, density and gradient checks across all fits: 18.

Limits of the conclusion

- One chain per fit; split R-hat is a within-chain diagnostic.
- Divergences: NCP 99, CP 707, post-hoc 55 / 43, online 6 / 12.
- Gradient counts are a compute proxy, not wall-clock speed.
- No reliable ranking or general speed advantage is established.
- These fits use the StanBlocks backend.

Refresh harness · committed comparison, controls and checks · source reproduction and audits.

Sources and reproduction

APPENDIX F

BRM examples and exports

- Feature atlas and PK example
- Warfarin source audit
- Wastewater structural port
- Grey-seal model and verification limits
- HSGP reproduction and recorded results
- Editable deck, references, and render command

Primary external sources

- brms nonlinear models, custom families, Stan functions
- Turing with differential equations
- Generable HSGP case study
- Stan reparameterization guide
- Reactant and probabilistic programming
- brms GPU support through OpenCL

Cited authors and prospective guests have not reviewed or endorsed this deck. Immutable scientific-source revisions and further references are in the README.